README

Lab 04: Phân loại cảm xúc IMDB

Lab này thử nghiệm một số cách biểu diễn văn bản để phân loại review phim IMDB thành hai nhãn `positive` và `negative`.

Thử nghiệm

- TF-IDF kết hợp Logistic Regression.
- Average Word2Vec kết hợp Logistic Regression.
- TF-IDF kết hợp Linear SVM.
- So sánh TF-IDF unigram với unigram và bigram.

Dữ liệu được chia thành tập train, validation và test. Các kết quả trong `results.csv` được tính trên tập validation.

Cách chạy

1. Cài các thư viện cần thiết:

   ```
   python3 -m pip install numpy pandas matplotlib scikit-learn
           gensim jupyter
   ```

2. Tải `IMDB Dataset.csv` và đặt trong thư mục `Lab04`.
3. Mở `classification.ipynb`, đổi đường dẫn CSV ở phần đầu notebook thành `IMDB Dataset.csv`.
4. Mở terminal tại thư mục `Lab04` và chạy:

   ```
   jupyter notebook classification.ipynb
   ```

Chạy các cell theo thứ tự từ trên xuống. Notebook sẽ tạo hoặc cập nhật `results.csv` và `confusion_matrix.png` trong thư mục đang chạy.

Kết quả chính

Mô hình	F1
TF-IDF + Logistic Regression	0.9042

Mô hình	F1
Average Word2Vec + Logistic Regression	0.8219
TF-IDF + Linear SVM	0.9045
TF-IDF unigram + bigram + Logistic Regression	0.9029

TF-IDF cho kết quả tốt hơn Average Word2Vec. Thêm bigram làm vocabulary tăng từ 92,647 lên 2,110,390 và thời gian train tăng từ khoảng 10.93 lên 60.81 giây, nhưng F1 giảm nhẹ.

Tệp trong thư mục

- `classification.ipynb`: kiểm tra dữ liệu, chạy thử nghiệm và đánh giá mô hình.
- `preprocessing.py`: chia dữ liệu và xử lý Average Word2Vec.
- `results.csv`: chỉ số đánh giá các mô hình.
- `confusion_matrix.png`: ma trận nhầm lẫn của mô hình baseline.
- `caculations.pdf`: tài liệu tính toán.
- `prediction.md`: dự đoán trước thí nghiệm.
- `error_analysis.md`: ví dụ phân loại đúng và sai.
- `reflection.md`: nhận xét và hướng phát triển tiếp theo.

6. Bài tiếp theo

Bài 1

$x = [1, 2, 0]$

$w = [0,5, -0,2, 0,4]$ $\quad b = 0,1$

+1 $z = w^T x + b$

$= [1, 2, 0] \cdot \begin{bmatrix} 0,5 \\ -0,2 \\ 0,4 \end{bmatrix} + 0,1$

$= 0,1 + 0,1 = 0,2$

+1 $P(y=1|x) = \sigma(z) = \frac{1}{1+e^{-0,2}} \approx 0,549$

bài 2

+1 $T = 0,8 \Rightarrow y = 1 \quad (0,8 < 0,72)$

+1 $T = 0,8 \Rightarrow y = 0 \quad (0,72 < 0,8)$

bài 3

$TP = 80, \ FN = 20, \ FP = 10, \ TN = 90$

$Precision = \frac{80}{80+10} = 0,889$

$Recall = \frac{80}{80+20} = 0,8$

$F_1 = \frac{2PR}{P+R} \approx 0,8421$

Bài 4

Vì sample model đều đoán đều là negative nên $TP = 0$

$\Rightarrow Recall = \frac{0}{0+FN} = 0$

$F_1 = 0$

Accuracy $\geq$ 90% nhưng model kém tốt vì dataset gần như chỉ có negative mà. Vì dataset gần như chỉ có negative nên phải xem thêm Precision, Recall, F_1, Confusion Matrix

Trong bài toán spam:

+1 FP : email bình thường bị nhận là spam

+1 FN : email spam nhận là bình thường

prediction(1)

Prediction trước experiment

Prediction 1

1. **Prediction:** TF-IDF + Logistic Regression có khả năng hoạt động tốt.
2. **Reason:** Vì TF-IDF bắt từ khóa đặc trưng tốt, còn LR xử lý tốt việc phân loại trên không gian từ vựng khổng lồ do TF-IDF tạo ra.
3. **Confidence:** High

Prediction 2

1. **Prediction:** Average Word2Vec không chắc chắn tốt hơn TF-IDF.
2. **Reason:** Average Word2Vec giữ thông tin ngữ nghĩa, nhưng việc lấy trung bình có thể làm mất các từ quan trọng và trật tự từ.
3. **Confidence:** Cao

Prediction 3

1. **Prediction:** Tăng vocabulary size không chắc chắn làm accuracy tăng.
2. **Reason:** Thêm quá nhiều từ vựng có thể kéo theo cả những từ hiếm và từ rác.
3. **Confidence:** High

Prediction 4

1. **Prediction:** Bỏ stopwords không chắc chắn làm performance tăng.
2. **Reason:** Một số stopwords có thể mang thông tin quan trọng cho classification, đặc biệt là các từ phủ định như “not”.
3. **Confidence:** High

Prediction 5

1. **Prediction:** Accuracy và F1 có thể cho kết luận khác nhau.
2. **Reason:** Accuracy đo tỷ lệ dự đoán đúng tổng thể, còn F1 cân bằng giữa Precision và Recall nên có thể rất khác biệt khi dataset mất cân bằng.
3. **Confidence:** High

preprocessing(1).py

```
import numpy as np
from sklearn.model_selection import train_test_split

# Split data
def split_data(df):
    X = df["review"]
    y = df["sentiment"].map({"negative": 0, "positive": 1})
    X_train, X_temp, y_train, y_temp = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp
    )
    return X_train, X_val, X_test, y_train, y_val, y_test

# Word2Vec
def tokenize(texts):
    return [text.lower().split() for text in texts]

def average_vectors(tokenized, model):
    return np.array([
        np.mean([model.wv[w] for w in tokens if w in model.wv], axis=0)
        if any(w in model.wv for w in tokens)
        else np.zeros(model.vector_size)
        for tokens in tokenized
    ])
```

classification.ipynb

11. Experiment 1 — Dataset inspection

```
import sys
sys.path.append("/kaggle/input/datasets/tngdngnguynl/lab04-
        preprocessing")

import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from gensim.models import Word2Vec
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score,
        recall_score, f1_score, ConfusionMatrixDisplay
from preprocessing import split_data, tokenize, average_vectors

path = "/kaggle/input/datasets/lakshmi25npathi/imdb-dataset-of-50k-
        movie-reviews/IMDB Dataset.csv"
df = pd.read_csv(path)
df["length"] = df["review"].str.split().str.len()

class_distribution =
        df["sentiment"].value_counts().rename_axis("Class").reset_index(name="Count")
class_distribution["Percentage"] = class_distribution["Count"] /
        len(df) * 100

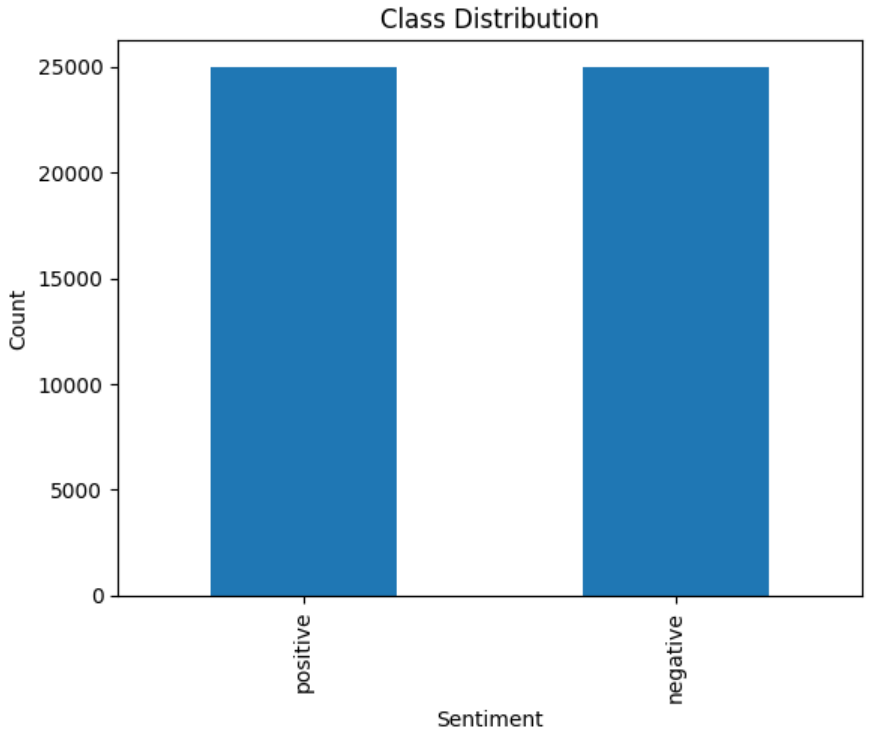
print("Samples:", len(df))
print("Classes:", df["sentiment"].nunique())
display(class_distribution)
print(df["length"].describe())
print("Duplicates:", df["review"].duplicated().sum())
print("Missing text:", df["review"].isna().sum())
print("Missing labels:", df["sentiment"].isna().sum())
```

```
Samples: 50000
Classes: 2
```

	Class	Count	Percentage
0	positive	25000	50.0
1	negative	25000	50.0

```
count    50000.000000
mean       231.156940
std        171.343997
min          4.000000
25%        126.000000
50%        173.000000
75%        280.000000
max       2470.000000
Name: length, dtype: float64
Duplicates: 418
Missing text: 0
Missing labels: 0
```

```
df["sentiment"].value_counts().plot(kind="bar")
plt.xlabel("Sentiment")
plt.ylabel("Count")
plt.title("Class Distribution")
plt.show()
```



Nhận xét

Dataset không bị imbalance vì positive và negative đều có 25,000 mẫu, mỗi lớp chiếm 50%.

12. Data leakage

Nếu fit TF-IDF trước khi chia dữ liệu, TF-IDF đã nhìn thấy từ và tần suất từ ở validation/test. Khi đó kết quả đánh giá không còn khách quan. Vì vậy chỉ fit TF-IDF trên tập train.

13. Core implementation

```
X_train, X_val, X_test, y_train, y_val, y_test = split_data(df)
print("Train:", len(X_train), "Validation:", len(X_val), "Test:",
        len(X_test))

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", LogisticRegression(max_iter=1000))
])
```

```
Train: 40000 Validation: 5000 Test: 5000
```

14. Experiment 2 — Baseline

```
start = time.time()
model.fit(X_train, y_train)
training_time = time.time() - start
```

```
y_pred = model.predict(X_val)
vocab_size = len(model.named_steps["tfidf"].vocabulary_)

baseline_result = {
    "Representation": "TF-IDF",
    "Classifier": "Logistic Regression",
    "Accuracy": accuracy_score(y_val, y_pred),
    "Precision": precision_score(y_val, y_pred),
    "Recall": recall_score(y_val, y_pred),
    "F1": f1_score(y_val, y_pred)
}

print("Training time:", training_time)
print("Vocabulary size:", vocab_size)
print(pd.Series(baseline_result))
```

```
Training time: 11.625940322875977
Vocabulary size: 92647
Representation                 TF-IDF
Classifier        Logistic Regression
Accuracy                        0.9032
Precision                     0.895294
Recall                          0.9132
F1                            0.904158
dtype: object
```

15. Experiment 3 — Representation comparison

Model B — Average Word2Vec + Logistic Regression

```
train_tokens = tokenize(X_train)
val_tokens = tokenize(X_val)

w2v = Word2Vec(train_tokens, vector_size=100, window=5, min_count=2)
X_train_w2v = average_vectors(train_tokens, w2v)
X_val_w2v = average_vectors(val_tokens, w2v)

w2v_model = LogisticRegression(max_iter=1000)
w2v_model.fit(X_train_w2v, y_train)
y_pred_w2v = w2v_model.predict(X_val_w2v)

w2v_result = {
    "Representation": "Avg Word2Vec",
    "Classifier": "Logistic Regression",
    "Accuracy": accuracy_score(y_val, y_pred_w2v),
    "Precision": precision_score(y_val, y_pred_w2v),
    "Recall": recall_score(y_val, y_pred_w2v),
    "F1": f1_score(y_val, y_pred_w2v)
}

print(pd.Series(w2v_result))
```

```
Representation           Avg Word2Vec
Classifier        Logistic Regression
Accuracy                        0.8272
Precision                       0.8151
Recall                          0.8464
F1                            0.830455
dtype: object
```

Model C — TF-IDF + Linear SVM

```
svm_model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", LinearSVC())
])

svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_val)
```

```
svm_result = {
    "Representation": "TF-IDF",
    "Classifier": "Linear SVM",
    "Accuracy": accuracy_score(y_val, y_pred_svm),
    "Precision": precision_score(y_val, y_pred_svm),
    "Recall": recall_score(y_val, y_pred_svm),
    "F1": f1_score(y_val, y_pred_svm)
}

print(pd.Series(svm_result))
```

```
Representation        TF-IDF
Classifier        Linear SVM
Accuracy              0.9034
Precision           0.894716
Recall                0.9144
F1                  0.904451
dtype: object
```

```
results = pd.DataFrame([baseline_result, w2v_result, svm_result])
display(results)
results.to_csv("results.csv", index=False)
```

	Representation	Classifier	Accuracy	Precision	Recall	F1
0	TF-IDF	Logistic Regression	0.9032	0.895294	0.9132	0.904158
1	Avg Word2Vec	Logistic Regression	0.8272	0.815100	0.8464	0.830455
2	TF-IDF	Linear SVM	0.9034	0.894716	0.9144	0.904451

16. Representation comparison — Nhận xét

TF-IDF cho kết quả tốt hơn Average Word2Vec trên IMDb (F1 0.9042 so với 0.8219). TF-IDF giữ được các từ quan trọng cho cảm xúc, còn Average Word2Vec lấy trung bình nên có thể làm mất thứ tự từ và giảm ảnh hưởng của các từ quan trọng.

17. Word order

Average Word2Vec không hiểu rõ vai trò của `not` trong `not good`. Khi lấy trung bình, `not` chỉ được cộng như một từ khác nên quan hệ phủ định giữa hai từ không được giữ lại.

18. Experiment 5 — n-gram features

```
bigram_model = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
    ("classifier", LogisticRegression(max_iter=1000))
])

start = time.time()
bigram_model.fit(X_train, y_train)
bigram_time = time.time() - start
y_pred_bigram = bigram_model.predict(X_val)

ngram_results = pd.DataFrame([
    ["Unigram", vocab_size, f1_score(y_val, y_pred), training_time],
    ["Uni + Bigram",
        len(bigram_model.named_steps["tfidf"].vocabulary_),
        f1_score(y_val, y_pred_bigram), bigram_time]
], columns=["Features", "Vocabulary", "F1", "Training time"])

display(ngram_results)
```

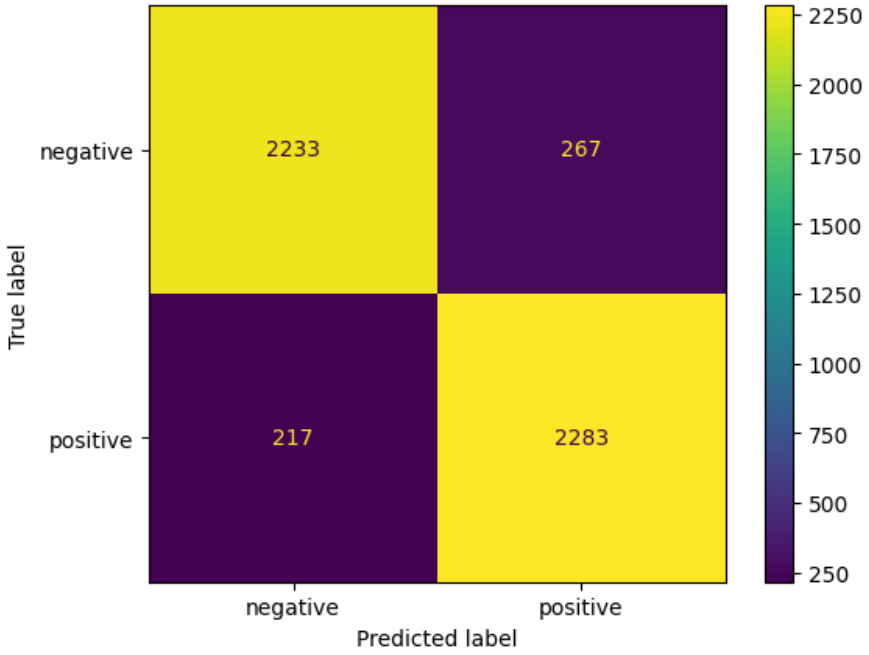
	Features	Vocabulary	F1	Training time
0	Unigram	92647	0.904158	11.625940
1	Uni + Bigram	2110390	0.902909	62.569832

Nhận xét

Bigram có thể giữ các cụm như `not good`, nhưng làm số lượng feature tăng rất mạnh. Trong kết quả này, vocabulary tăng từ 92,647 lên 2,110,390 và thời gian train tăng từ 10.93s lên 60.81s, trong khi F1 giảm nhẹ từ 0.9042 xuống 0.9029.

20. Confusion matrix

```
ConfusionMatrixDisplay.from_predictions(
    y_val, y_pred, display_labels=["negative", "positive"]
)
plt.savefig("confusion_matrix.png", bbox_inches="tight")
plt.show()
```



Nhận xét

Negative bị nhầm sang positive nhiều hơn: 267 mẫu, so với 217 mẫu positive bị nhầm sang negative. Vì false positive nhiều hơn false negative nên Precision phản ánh rõ vấn đề này; Precision cũng thấp hơn Recall.

21. Error analysis

```
labels = {0: "negative", 1: "positive"}
error_df = pd.DataFrame({
    "Text": X_val.values,
    "True": [labels[x] for x in y_val.values],
    "Predicted": [labels[x] for x in y_pred]
})

correct_samples = error_df[error_df["True"] ==
        error_df["Predicted"]].sample(3, random_state=42)
incorrect_samples = error_df[error_df["True"] !=
        error_df["Predicted"]].sample(3, random_state=42)
```

```
display(correct_samples)
display(incorrect_samples)
```

	Text	True	Predicted
1728	I, for one, absolutely loved this movie. ...	positive	positive
1568	This movie is just another average action flic...	negative	negative
4436	This movie scared heck out of me when I was ju...	positive	positive

	Text	True	Predicted
4263	"House of Dracula" isn't all that bad of a fil...	positive	negative
4930	The NYT review says that Sigourney Weaver's ch...	positive	negative
1887	I missed the beginning but I did see most of i...	negative	positive

22. Inspect model weights

```
tfidf = model.named_steps["tfidf"]
clf = model.named_steps["classifier"]
features = np.array(tfidf.get_feature_names_out())
weights = clf.coef_[0]

positive_idx = np.argsort(weights)[-10:][::-1]
negative_idx = np.argsort(weights)[:10]

top_positive = pd.DataFrame({"Feature": features[positive_idx],
        "Weight": weights[positive_idx]})
top_negative = pd.DataFrame({"Feature": features[negative_idx],
        "Weight": weights[negative_idx]})

display(top_positive)
display(top_negative)
```

	Feature	Weight
0	great	8.806805
1	excellent	7.430747
2	best	5.634857
3	perfect	5.611981
4	wonderful	5.141145
5	amazing	5.126353
6	loved	4.814875
7	enjoyed	4.594796
8	fun	4.424613
9	brilliant	4.332755

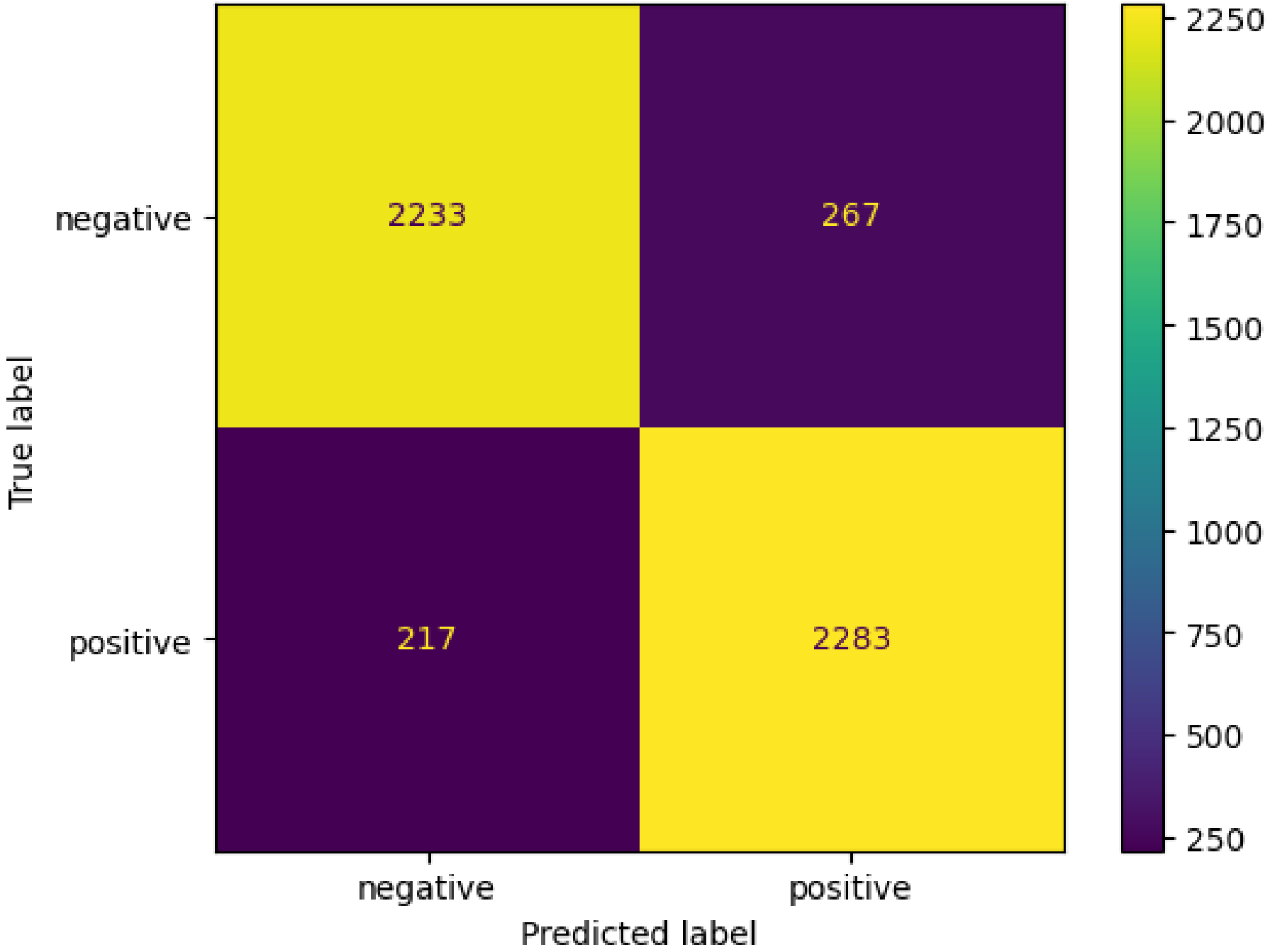
	Feature	Weight
0	worst	-11.054083
1	bad	-9.307337
2	awful	-8.259308
3	waste	-8.010474
4	boring	-7.201748
5	poor	-6.434099
6	terrible	-6.364630
7	nothing	-5.665740
8	worse	-5.346936
9	dull	-5.273588

Nhận xét

Các feature nhìn chung hợp lý với bài toán sentiment. Những từ như `great`, `excellent`, `amazing` nghiêng về positive, còn `worst`, `bad`, `awful`, `boring` nghiêng về negative.

results.csv

Experiment	Representation	Features	Classifier	Vocabulary	Training time	Accuracy	Precision	Recall	F1
Baseline / Model A / N-gram Unigram	TF-IDF	Unigram	Logistic Regression	92647	10.925874	0.9032	0.895294	0.9132	0.904158
Model B	Avg Word2Vec		Logistic Regression			0.8188	0.808198	0.836	0.821864
Model C	TF-IDF	Unigram	Linear SVM			0.9034	0.894716	0.9144	0.904451
N-gram Uni + Bigram	TF-IDF	Unigram + Bigram	Logistic Regression	2110390	60.812531				0.902909


2233
267
217
2283
negative
positive
True label
negative
positive
Predicted label
2250
2000
1750
1500
1250
1000
750
500
250

error_analysis(1)

Error Analysis

1. Correctly Classified

Text	True	Predicted
I, for one, absolutely loved this movie...	positive	positive
This movie is just another average action flick...	negative	negative
This movie scared heck out of me when I was just...	positive	positive

2. Incorrectly Classified

Text	True	Predicted	Analysis
"House of Dracula" isn't all that bad of a film...	positive	negative	Câu có phủ định, dễ làm model hiểu sai.
The NYT review says that Sigourney Weaver's character...	positive	negative	Câu dài và có nhiều ý nên khó xác định cảm xúc chính.
I missed the beginning but I did see most of it...	negative	positive	Nội dung khá mơ hồ, có cả ý trung tính và tiêu cực.

reflection(1)

Reflection

Qua bài lab, em nhận thấy cách biểu diễn văn bản có ảnh hưởng đến kết quả phân loại. Với bộ dữ liệu IMDB, TF-IDF cho kết quả tốt hơn Average Word2Vec. Khi thêm bigram, số lượng đặc trưng tăng mạnh nhưng F1 gần như không cải thiện, trong khi thời gian train lâu hơn nhiều. Điều này cho thấy thêm nhiều đặc trưng chưa chắc làm mô hình tốt hơn.

Một số câu có phủ định hoặc nhiều ý vẫn dễ bị dự đoán sai vì các cách biểu diễn hiện tại chưa nắm bắt tốt thứ tự và ngữ cảnh của từ. Đây là lý do cần chuyển sang các mô hình có khả năng xử lý chuỗi và ngữ cảnh tốt hơn như RNN, LSTM hoặc GRU.